Extending QA with Claude Code: a Senior Manual QA case study

I am a Senior Manual QA Engineer, not an automation engineer. This case study shows how I used Claude Code to extend my QA capabilities into automation, diagnostics and repeatable QA operations — without treating AI-generated code as a replacement for QA or engineering judgement.

E-commerce QA — web, Android & iOS
13 years in QA · state as of October 2026

Anonymized for portfolio use: client name, internal domains and Slack channel names replaced. Architecture, counts and gaps are unedited.

Oleksandr Kukhtinov

What this shows — and what it does not

What it shows

- How I decide what is worth testing, what is worth automating, and what has to stay manual.
- That I can specify a test oracle that proves something, rather than an assertion that a page looks right.
- That I can turn tribal QA knowledge — ownership, request diagnosis, triage — into procedures anyone can run.
- That I can direct an AI coding tool to a working result, with safety, scope and review under my control.

What it does not show

- Hands-on automation engineering. I did not write this framework by hand, and I do not claim Selenium, Playwright or Appium as skills of mine.
- Code ownership. The code is reviewed by me, not authored by me.
- Deep automated regression. The suites are a proof of concept, smoke-level by design; deeper testing stays manual and mine.
- A finished product. Several parts are unverified, and slide 11 names each one.

The honest summary

- The QA engineering is mine, the implementation is Claude Code's, and the review and verification are mine again.
- **Where that judgement comes from:** 13 years in QA — e-commerce, US wealth management, ad-tracking, real estate.
- 2015–2022: a US wealth-management portal — storage, reconciliation and validation of financial data and reporting; API responses reconciled against the UI and the database in SQL.
- 2022–present: a UK e-commerce marketplace, admin platform and mobile apps; release cycles from ten days to weekly; QA processes built from scratch.

The QA flow behind every part of this

Nothing here started from “let's automate something”. Each piece of work is one pass through the same chain.

01
Requirement
What the feature actually promises the customer

02
Risk analysis
What failing here costs, and how likely it is to fail quietly

03
Testability
Can this be observed at all? If not, that is the first finding

04
Test design
Scenarios, data, guest vs signed-in paths, negative cases

05
Automation candidate
Only what is repetitive, stable and cheap to keep alive

06
Test oracle
What proves pass or fail — server response, not appearance

07
Observability
Requests, traces and app version captured on every run

08
Evidence
Screenshots, network log and build version saved automatically

09
Failure diagnosis
requestId to trace to the failing span, drafted for a human

10
Developer feedback
Owning team named, ticket drafted, a person sends it

Designed and judged by me

Carried by Claude Code

ONE PASS PER PIECE OF WORK

> Steps 1–6 and 10 are the job I have done for 13 years. Steps 7–9 are the part that used to be manual archaeology across three systems, and are the part Claude Code made repeatable.

Who did what

The question a reviewer should not have to ask.

AREA	MY CONTRIBUTION	CLAUDE CODE	HOW IT SHOWS UP IN THE PACKAGE
QA architecture	**Designed**	Implemented	Two parts with no shared code or state: suites and skills
Test strategy	**Decided**	—	What is automated, what stays manual, what is out of scope entirely
Test scenarios	**Written by me — steps, data, expected results**	Translated into code	24 smoke-level Android checks — 17 as guest, 7 as signed-in customer
Test oracles & assertions	**Designed**	Implemented	Server responses watched alongside the UI on every check
Automation code	**Scoped, reviewed and validated**	Authored by Claude Code	TypeScript suites and the shared scaffold reused across iOS
Safety & compliance rules	**Designed**	Implemented as guards	Test environment only, no auto-accept, no credentials in the package
Skills & workflows	**Designed the workflow**	Implemented	Six on-demand diagnostic skills; none scheduled, none touching production
Documentation	**Reviewed**	Authored by Claude Code	Ten printed guides, plus seven guards that fail when a guide drifts
Verification	**Performed personally**	—	Stated per item on slide 11, including what remains unverified

I am not the author of the automation code. I am the author of what it checks, why it checks that, and what counts as proof.

What problem each part actually solves

Three long-standing problems in how the team tested — and what changed in each.

AUTOMATED SUITES

Before: every release was checked by hand on three platforms, which could not keep up with the release pace — and a tester could see a page look completely right while the server behind it returned errors.

Now: 24 smoke-level checks on Android (17 guest, 7 signed-in) and the same 24 on iOS from an identical scaffold; web runs on the same base and is next to reach parity, though so far it only launches the site. Scope is deliberately smoke: installation, launch, key screens and network-log verification — but each check watches the server's replies, so a hidden error fails it instead of passing silently.

QA value: when something breaks, the evidence is already collected — which build, which step, which request. No “can you reproduce it?” round trip.

DIAGNOSTIC SKILLS

Before: “whose team owns this?” and “what actually broke behind this error?” meant manual digging across chat, monitoring and the tracker — and usually by the same two or three people who knew where to look.

Now: six on-demand procedures, triggered in plain language, that do the digging and return a draft: owning team and channel, the failing service behind an error, a ticket summary, new store reviews, squad decision history, or which feature flags a customer actually had.

QA value: the knowledge stops living in one person's head. A new joiner gets the same shortcuts on day one, and an answer no longer waits for the right colleague to be online.

EVERYDAY SKILLS

Before: small recurring questions — what did I miss in chat, what should I know before today's meetings, what is the flag rollout by brand and platform, and how does a Mac-built script run on a colleague's Windows machine — were each too small to justify a tool, so they were answered by hand, again and again.

Now: four importable skills, one per question. Imported once, then asked for in plain language. No schedule, no configuration, no new system to maintain.

QA value: removes recurring overhead from the team without adding anything else that someone has to own.

> What ties the three together: knowledge that lived in one person's head becomes something anyone can request from day one — with no bottleneck and no action taken against production.

The decisions that shaped it — and the ones I refused

Risk-based scoping is the part that does not come from a tool. These are the calls I made before any code existed.

What I chose to automate

- Repetitive pre-release paths across three platforms — high frequency, stable, expensive by hand.
- The guest and signed-in journeys split deliberately: they fail differently, so they are scoped and reported separately.
- The archaeology steps — ownership lookup, request tracing, ticket summarising — because they are mechanical and were bottlenecked on two or three people.

What I kept with a human, on purpose

- Deciding what is worth testing at all, and judging whether a failure is a real defect.
- Anything touching production or an account other than the test one.
- Release sign-off, and any judgement I would put my own name to.
- Creating tickets and test cases: the skills draft, a person confirms and sends.

Testability treated as a finding, not an obstacle

- If a behaviour cannot be observed from outside, that is reported as a gap in the product, not worked around in the test.
- Suites are non-destructive by default; a mutating scenario copies the existing pattern and cleans up after itself in `finally`.

Kept deliberately boring

- No shared code or state between the ten skill modules (six diagnostic, four everyday). Any one can be deleted without breaking the other nine.
- Nothing on a schedule. Every skill runs only when a person asks for it.

Test oracles and evidence, not green checkmarks

A UI assertion tells you a page rendered. It does not tell you the system was correct. This is the distinction the whole suite is built on.

What the oracle actually checks

- The server's responses are watched alongside the UI on every check, so a failing request fails the test instead of passing behind a correct-looking page.
- An oracle has to prove it is not blind: zero requests observed raises a loud warning rather than a quiet pass. A silent green is treated as a defect in the test.
- Negative and error paths are asserted on the response, where the product's real behaviour lives.

What every run leaves behind

- Screenshots at each step, tied to the exact build under test.
- The network log for the run, so the failing request is already captured.
- The app version, so a failure is attributable to a build rather than to a memory of one.
- A run summary, rotated so evidence does not grow without limit.

> Why a manual QA cares about this: it removes the most expensive part of a bug report — reproducing the failure to convince someone it happened. The proof is collected before anyone asks for it.

From a failure to the right developer

Every diagnostic skill follows the same shape. Only the tool at step 4 changes. Worked example: a requestId from a failed run.

STEP 1
Ask
A requestId in hand, asked for in plain language

STEP 2
Conventions load
Project conventions load automatically in every conversation — no setup

STEP 3
Skill runs
A registered procedure, shown and confirmed before it starts

STEP 4
Subagent digs
An isolated context walks the trace; none of it touches the parent conversation

STEP 5
Draft, then a human
Failing span named, ticket drafted but not created. A person reviews before anything is sent

Same shape, different tool at step 4

- Ownership lookup swaps in the chat connector: which team owns this error, which channel, who to tag.
- Ticket triage swaps in the issue tracker, read-only.
- Customer flag lookup swaps in the flag CLI, and always answers as a lower bound — never as a claim of absence.

Why this is a QA artefact, not a chatbot

- It ends at a draft on purpose. Nothing is created, sent or closed without a person.
- The routing rule — error in the product under test vs. problem in the tooling itself — is written down, so escalation stops depending on who is asked.

The safety rules are mine, and they are not negotiable

These were set before the first line of generated code and were never relaxed for convenience.

Environment and blast radius

- Test environment only, never production.
- Never an account other than the designated test account — which is live, with real data, so suites stay non-destructive.
- A mutating test cleans up after itself in `finally`, with a cleanliness check afterwards.
- All six diagnostic skills run on demand. Nothing in the package runs on a schedule.

Control of the AI tool

- No auto-accept and no skipped permissions. Every command is confirmed by a person.
- The tool does not replace review: a human checks any output before it is acted on.
- Work account only — no personal licences, no free trials, no personal projects.

Credentials

- The package ships with zero tokens and zero secrets. Each module that needs one provisions its own.
- One token file inside the module that needs it, never a synced folder, `chmod 600`, minimum scope, an expiry where one is available.
- Secrets never go into chat, calls or shared documents. A leaked token is revoked, not quietly abandoned.

The package tells on itself

- Seven automated guards fail loudly the moment a guide, launcher or module drifts from reality.
- The handover-prep script reports what is left in the package — regenerable run logs, and zero credentials.
- Gaps are stated as gaps. A false green is worse than an honest hole.

What Claude Code replaced — and what it did not

The useful version of this claim is narrow.

It replaced

- The gap between knowing what a test should assert and being able to write the code that asserts it.
- Weeks of scaffolding: the iOS suite reuses the Android scaffold rather than being rebuilt.
- Writing and maintaining ten guides that stay consistent with the code, enforced by guards rather than by discipline.
- The repetitive digging in ownership lookup, trace walking and ticket summarising.

It did not replace

- Deciding what is worth testing. A generated suite will happily cover the wrong things thoroughly.
- Specifying the oracle. The model does not know what “correct” means for this business.
- Judging whether a failure is a defect, a data problem or a bad test.
- Accountability. I reviewed every generated artefact before it entered the package, and I am the one who has to defend it.

> Stated plainly, because it matters more than any of the above: generated code is not evidence of automation expertise. It is evidence that a QA engineer with a clear specification can now produce working tooling. Those are different claims, and I am only making the second one.

Where each part actually stands

Per item, as of October 2026. Nothing here is upgraded to sound better than it is.

ITEM	STATUS	WHAT THAT MEANS EXACTLY
Android suite	RUN	24 smoke-level checks executed against the test environment; re-run after the package was restructured.
iOS suite	RUN	The same 24 smoke-level checks from an identical scaffold; re-run after the restructure.
Web suite	PARTIAL	Built on the same base, but so far it has only opened the site. Planned: 24 smoke checks, as on mobile. Not at parity, and I do not present it as such.
PowerShell tooling	NOT VERIFIED	Written and reviewed. Never executed on a real Windows machine.
Six diagnostic skills	RUN	Each exercised against its own tool. Every one ends at a draft, so a wrong answer is caught by the reviewer, not shipped.
Seven guards	RUNNABLE CHECK	Anyone can run them; all seven must exit 0. This is the check I hand over rather than an assurance.
Test-case creation in Xray	WRITTEN, NOT RUN LIVE	Implemented as a skill: dry run first, then the plan hash and the host confirmed by a person. Tested offline; not yet run against a live Xray. Skill evals not run.
Bug creation from the tracker	WRITTEN, NOT RUN LIVE	Implemented as a skill: one issue per human confirmation. Create and edit rights are stated by the author; not yet run against the live tracker. Skill evals not run.

If a first run fails on one of these, it is the package, not the person running it. That sentence is in the handover guide too.

What I would do differently — and what I would keep

If I started again

- Build the server-side oracle and the evidence system in week one, not after the suites existed. Retrofitting it meant reworking every case already written.
- Settle the folder naming convention before the bootstrap scripts started auto-detecting two variants. The auto-detection is harmless; it is the symptom of a decision deferred too long.
- Request observability and log-search access in week one. The approval lag is the same whether you ask early or late, so there was no reason to wait.

What I would keep exactly as it is

- Test environment only, confirm before every run, no auto-accept. That friction costs something on every single run, and I would pay it again.
- No shared code or state between modules. Any one of the ten skills can be deleted without breaking the other nine.
- Stating gaps as gaps instead of quietly routing around them. A false green is worse than an honest hole, and slide 11 is the proof I mean it.

None of this changes the architecture. It changes the order I would build it in.

APPENDIX

What is actually in the package

Inventory only. Setup instructions, access prerequisites and escalation paths live in the handover guides, not in this deck.

PART	WHAT IT IS	NOTES
Automated suites	Web, Android and iOS suites against a dedicated test environment. TypeScript, shared scaffold.	24 smoke-level checks per mobile platform — installation, launch, key screens, network-log verification. Non-destructive by default.
Six diagnostic skills	Ownership lookup, request diagnosis, ticket summary, store-review digest, squad decision mirror, customer flag lookup.	Self-contained, no shared state, all on demand.
Four skills	Chat digest, meeting brief, feature-flag status, cross-platform adaptation notes for running Mac-built scripts on Windows.	One importable file each. Nothing to register or schedule.
Scripts and guards	Two document builders, a skill packager, and seven guards.	Guards fail when documentation drifts from the code.
Ten printed guides	One per diagnostic skill module, two for the suites, one for the everyday skills, one for the package root.	Written for a person who does not use a terminal.
Conventions & decision log	Project conventions loaded into every session, plus a numbered log of decisions.	On any conflict, the decision log wins.

Questions

Oleksandr Kukhtinov

Senior Manual QA Engineer · 13 years

AI-assisted QA: automation, diagnostics and repeatable QA operations

A sanitized case study built from a real system. Client identity, internal domains and Slack channel names have been replaced for portfolio use; the architecture, the counts and the gaps are unedited. I did not write the automation code by hand, and I am happy to walk through any decision in it.